



1. Core conceptual shift

Rectified Flow (your current code)

You learn a velocity field:

$$v_{\theta}(x, t) \approx x_1 - x_0$$

- deterministic path (ODE)
 - regression on velocity
 - Euler integration
-

DDPM (diffusion model)

You learn noise prediction:

$$\epsilon_{\theta}(x_t, t) \approx \epsilon$$

- stochastic process (SDE)
- forward noising process
- reverse denoising steps

2. Key mapping (RF → DDPM)

Component	Rectified Flow	DDPM
Target	$x_1 - x_0$	noise ϵ
Input	interpolated x_t	noisy x_t
Process	linear interpolation	Gaussian diffusion
Dynamics	ODE	SDE
Loss	MSE(v)	MSE(ϵ)
Sampling	Euler ODE	iterative denoising

🧠 DDPM VERSION (same architecture, different math)

Below is your Rectified Flow code rewritten as DDPM.

🔄 1. Model (UNCHANGED)

Python

📄 Copy

```
model = UNetDDPM().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

✓ Same architecture as RF



2. TRAINING LOOP (THIS IS WHERE IT CHANGES)



RF version (what you had)

Python

Copy

```
x0 = torch.randn_like(x1)
t = torch.rand(B)

xt = (1 - t) * x0 + t * x1
v_target = x1 - x0
loss = MSE(v_pred, v_target)
```



DDPM version (NEW)

Python

Copy

```
def cosine_beta_schedule(T):
    steps = T + 1
    x = torch.linspace(0, T, steps)
    alphas_cumprod = torch.cos((x / T + 0.008) / 1.008 * torch.pi / 2)
    alphas_cumprod = alphas_cumprod / alphas_cumprod[0]
    betas = 1 - (alphas_cumprod[1:] / alphas_cumprod[:-1])
    return torch.clip(betas, 0.0001, 0.02)
```

Training step (DDPM core)

Python

 Copy

```
T = 1000
betas = cosine_beta_schedule(T).to(device)
alphas = 1.0 - betas
alphas_cumprod = torch.cumprod(alphas, dim=0)

def train_step(x0):
    x0 = x0.to(device)

    B = x0.shape[0]
    t = torch.randint(0, T, (B,), device=device)

    noise = torch.randn_like(x0)

    # diffusion scaling
    a_t = alphas_cumprod[t].view(B,1,1,1)

    x_t = torch.sqrt(a_t) * x0 + torch.sqrt(1 - a_t) * noise

    noise_pred = model(x_t, t.float() / T)

    loss = ((noise_pred - noise) ** 2).mean()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    return loss.item()
```

🔴 DFI 1 — DATA CORRUPTION PROCESS

RF:

Python

```
x_t = (1 - t) x_0 + t x_1    # linear interpolation
```

DDPM:

Python

```
x_t = sqrt(a_t) x_0 + sqrt(1-a_t) ε
```

- ✓ RF: deterministic path
- ✓ DDPM: stochastic Gaussian noise process

🔴 DFI 2 — TARGET FUNCTION

RF:

Python

```
v_target = x_1 - x_0
```

DDPM:

Python

```
noise_target = ε
```

- ✓ RF learns **direction field**
- ✓ DDPM learns **noise removal**



🔴 DFI 3 — TIME REPRESENTATION

RF:

Python

```
t ∈ [0,1] continuous
```

DDPM:

Python

```
t ∈ {0,1,...,T} discrete
```

🔴 DFI 4 — LOSS FUNCTION

RF:

Python

```
MSE(v_pred, v_target)
```

DDPM:

Python

```
MSE(ε_pred, ε)
```

✓ RF = velocity regression

✓ DDPM = denoising regression

DFI 5 — FORWARD PROCESS

RF:

- no diffusion process
- just interpolation

DDPM:

- fixed Markov chain:

Python

```
q(x_t | x_{t-1}) = Gaussian
```


DFI 6 — SAMPLING (VERY IMPORTANT)

RF sampling:

Python

```
x = noise
x = x + dt * v(x,t)
```

ODE solver

DDPM sampling:

Python

```
for t in reversed(range(T)):
    noise_pred = model(x_t, t)
    x_{t-1} = denoise_step(x_t, noise_pred)
```

Stochastic reverse process

DFI 7 — MATHEMATICAL OBJECTIVE

Method

Learns

RF

vector field (ODE)

DDPM

score / noise (SDE)

4. SAME ARCHITECTURE, DIFFERENT MEANING

Important subtle point:

👉 Your UNetDDPM stays unchanged

But interpretation changes:

RF UNet	DDPM UNet
outputs velocity	outputs noise
learns transport	learns denoising

5. Minimal mental model

Rectified Flow:

“Move points directly toward data”

DDPM:

“Remove noise step-by-step”

1. DDPM forward process

Discrete diffusion process:

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

This defines a Markov chain:

- data $\rightarrow$ noise
 - via small Gaussian perturbations
-

2. Reverse process (learned model)

$$p_\theta(x_{t-1} | x_t)$$

The model learns to invert the diffusion process step by step.

3. Continuous-time limit

As the number of steps $T \rightarrow \infty$, DDPM becomes an SDE:

$$dx = f(x, t) dt + g(t) dW_t$$

Key point:

- dW_t = stochastic Brownian motion
- still inherently random

4. Rectified Flow removes stochasticity

Rectified Flow replaces the SDE with a deterministic ODE:

$$\frac{dx}{dt} = v(x, t)$$

So:

- DDPM → stochastic dynamics
 - RF → deterministic transport
-

5. DDPM training target (noise prediction)

$$\epsilon_{\theta}(x_t, t) \approx \epsilon$$

Model learns to predict Gaussian noise added to data.

6. RF training target (velocity prediction)

$$v_{\theta}(x_t, t) \approx x_1 - x_0$$

Model learns the **transport direction** between paired samples.

7. Key interpolation difference

DDPM forward process:

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$$

Rectified Flow:

$$x_t = (1 - t)x_0 + tx_1$$



🔥 8. Conditional expectation bridge

DDPM learns a denoising function:

$$\mathbb{E}[\epsilon \mid x_t]$$

Rectified Flow instead learns a deterministic velocity field:

$$v(x, t) = \mathbb{E}[x_1 - x_0 \mid x_t]$$

⚡ 9. Core limiting intuition

DDPM can be written (in continuous form) as:

$$\frac{dx}{dt} = f(x, t) - \frac{1}{2}g(t)^2 \nabla_x \log p_t(x)$$

Rectified Flow corresponds to the regime where:

- trajectories are **straightened**
- stochasticity becomes negligible
- drift becomes deterministic

🧠 10. Final intuition

DDPM is a stochastic diffusion process; Rectified Flow is the deterministic limit obtained when those stochastic trajectories are progressively straightened into transport paths.



1. Score matching → velocity field (analytic bridge)

We start from the continuous diffusion / score model.

Step 1: score-based SDE

$$dx = f(x, t) dt + g(t) dW_t$$

The model learns the **score**:

$$s_\theta(x, t) \approx \nabla_x \log p_t(x)$$

Step 2: reverse-time dynamics

The reverse SDE is:

$$dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t) d\bar{W}_t$$

So the **drift term** is:

$$u(x, t) = f(x, t) - g(t)^2 \nabla_x \log p_t(x)$$

Step 3: deterministic probability flow ODE

There exists an equivalent ODE with identical marginals:

$$\frac{dx}{dt} = f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x)$$

This is already **deterministic**.

Step 4: key substitution

Now define the **velocity field**:

$$v(x, t) \equiv -\frac{1}{2} g(t)^2 \nabla_x \log p_t(x)$$

So:

$$\frac{dx}{dt} = f(x, t) + v(x, t)$$

Step 5: Rectified Flow insight

Rectified Flow replaces score estimation with:

$$v(x, t) \approx \mathbb{E}[x_1 - x_0 \mid x_t]$$

So instead of:

- estimating gradients of log-density

we directly learn:

- transport velocity



Key conclusion (Score → RF)

$$\nabla_x \log p_t(x) \implies \mathbb{E}[x_1 - x_0 \mid x_t]$$



score field → transport field

Step 5: Rectified Flow insight

Rectified Flow replaces score estimation with:

$$v(x, t) \approx \mathbb{E}[x_1 - x_0 \mid x_t]$$

So instead of:

- estimating gradients of log-density

we directly learn:

- transport velocity



Key conclusion (Score $\rightarrow$ RF)

$$\nabla_x \log p_t(x) \implies \mathbb{E}[x_1 - x_0 \mid x_t]$$



score field $\rightarrow$ transport field

2. Geometric / Optimal Transport view

Now the more intuitive (and deeper) view.

Step 1: think of distributions

We have:

- data distribution p_0
- noise distribution p_1

We want to transport mass from $p_0 \rightarrow p_1$

Step 2: stochastic diffusion (DDPM)

DDPM generates many random paths:

- particles diffuse
- trajectories cross
- motion is noisy

This corresponds to a **stochastic coupling**

Step 3: Rectified Flow goal

RF instead learns:



2. Geometric / Optimal Transport view

Now the more intuitive (and deeper) view.

Step 1: think of distributions

We have:

- data distribution p_0
- noise distribution p_1

We want to transport mass from $p_0 \rightarrow p_1$

Step 2: stochastic diffusion (DDPM)

DDPM generates many random paths:

- particles diffuse
- trajectories cross
- motion is noisy

This corresponds to a **stochastic coupling**

Step 3: Rectified Flow goal

RF instead learns:

a **deterministic flow map** T_t

such that:

$$x_t = T_t(x_0)$$



2. Geometric / Optimal Transport view

Now the more intuitive (and deeper) view.

Step 1: think of distributions

We have:

- data distribution p_0
- noise distribution p_1

We want to transport mass from $p_0 \rightarrow p_1$

Step 2: stochastic diffusion (DDPM)

DDPM generates many random paths:

- particles diffuse
- trajectories cross
- motion is noisy

This corresponds to a **stochastic coupling**

Step 3: Rectified Flow goal

RF instead learns:

▮ a **deterministic flow map** T_t

such that:

$$x_t = T_t(x_0)$$



$$x_t = T_t(x_0)$$

and:

$$T_0 = \text{identity}, \quad T_1 \sim p_1$$

Step 4: velocity field interpretation

The flow satisfies:

$$\frac{dx_t}{dt} = v(x_t, t)$$

So:

- each particle follows a **unique trajectory**
 - no randomness in path
-

Step 5: optimal transport connection

The ideal transport solves:

$$\min_T \mathbb{E} \|x - T(x)\|^2$$

This is **Wasserstein-2 optimal transport**

Rectified Flow approximates this by learning:

$$v(x, t) \approx \text{optimal transport direction}$$



Step 6: straight-line ideal case

Step 6: straight-line ideal case

If transport is optimal:

$$x_t = (1 - t)x_0 + tx_1$$

Then velocity is constant:

$$v(x, t) = x_1 - x_0$$

👉 this is exactly RF training target

🔥 3. Unified intuition (both views meet)

View	DDPM	Rectified Flow
Probabilistic	SDE	ODE
Object learned	score $\nabla \log p$	velocity field
Geometry	noisy diffusion paths	straight transport paths
Time evolution	stochastic	deterministic
Goal	denoise	transport

Final takeaway

From score view:

- RF replaces gradient estimation of log-density with direct learning of conditional displacement.

From geometry view:

- RF replaces random diffusion paths with straight optimal transport trajectories.